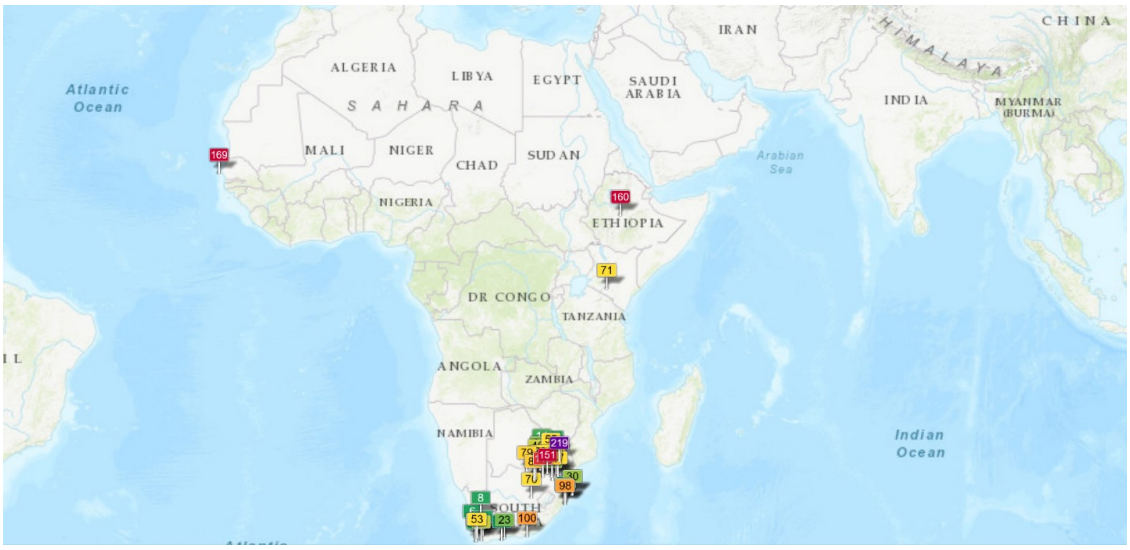
# tanzania-air-quality-27112023

December 2, 2023

**Project Introduction:** .
Embarking on a journey through Tanzania's air reveals more than just data points; it unveils a narrative of environmental rhythms. In the pulsating heartbeats of time series, we unravel the nuanced tale of air quality, bridging the gap between data and the breath of a nation. Join me in exploring the unseen symphony of Tanzania's air, where each spike and dip echoes a story of environmental impact and resilience.

[1]:
```
from PIL import Image

intro_image = Image.open(r"C:\Users\23162342\Desktop\POS Reports\python␣
↪timeseries project\Air_quality_map.jpg")
intro_image
```

[1]:



**About Data:** .
The dataset utilized in this study contains comprehensive metrics to evaluatePM2.5 reading of the Air quality inAir quality Tanzaniaes. Key variables"sensor_id", "sensor_type","location","lat","lon","timestamp","P1","durP1","ratioP1","P2","durP2","ratioP2".n." This rich dataset provides a unique opportunitypredict the Air quality in the state till dateevels.

**Data Source** .
The dataset for this study was sourced from africaopendata.org, updated last at May 30, 2018.

**Objectives:** .
Explaratory Data analysis(EDA): to understand what constitute the data, how rich and how reliable, to also check for anomalies and how they have affected the dataset, the distributions likewise the trend over time.
Predictive Modeling: To develop a machine learning model that can predict the country's future Air quality.

[2]:
```
# Import libraries here
import time
from datetime import datetime, timedelta
import pytz
import warnings

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import plotly.express as px
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
warnings.filterwarnings("ignore")
```

[3]:
```
#read the file in a dataframe
df=pd.read_csv(r"C:\Users\23162342\Desktop\POS Reports\python timeseries␣
↪project\Temekeairquality.csv")
```

**Date inspection:** .

[4]:
```
df.head()
```

[4]:

| | sensor_id | sensor_type | location | lat | lon | \ |
|---|---|---|---|---|---|---|
| 0 | 29 | SDS011 | 11 | -6.818 | 39.285 | |
| 1 | 29 | SDS011 | 11 | -6.818 | 39.285 | |
| 2 | 29 | SDS011 | 11 | -6.818 | 39.285 | |
| 3 | 29 | SDS011 | 11 | -6.818 | 39.285 | |
| 4 | 29 | SDS011 | 11 | -6.818 | 39.285 | |

| | timestamp | P1 | durP1 | ratioP1 | P2 | durP2 | ratioP2 |
|---|---|---|---|---|---|---|---|
| 0 | 2017-12-01T00:00:44.094494+00:00 | 6.03 | NaN | NaN | 4 | NaN | NaN |
| 1 | 2017-12-01T00:03:13.870148+00:00 | 4.93 | NaN | NaN | 3.9 | NaN | NaN |
| 2 | 2017-12-01T00:05:43.505681+00:00 | 4.30 | NaN | NaN | 3.9 | NaN | NaN |
| 3 | 2017-12-01T00:08:13.116753+00:00 | 8.70 | NaN | NaN | 4.3 | NaN | NaN |
| 4 | 2017-12-01T00:10:42.711977+00:00 | 6.97 | NaN | NaN | 4.6 | NaN | NaN |

[5]:
```
df.describe()
```

[5]:
```
        sensor_id      location           lat           lon            P1  \
count     58994.0  58994.000000  58994.000000  58994.000000  58994.000000
mean         29.0     11.575042     -6.822391     39.284059     29.506995
std           0.0      2.448446      0.018697      0.004007     47.369703
min          29.0     11.000000     -6.902000     39.267000      0.800000
25%          29.0     11.000000     -6.818000     39.285000     10.730000
50%          29.0     11.000000     -6.818000     39.285000     20.930000
75%          29.0     11.000000     -6.818000     39.285000     30.100000
max          29.0     22.000000     -6.818000     39.285000   1054.570000

        durP1  ratioP1  durP2  ratioP2
count     0.0      0.0    0.0      0.0
mean      NaN      NaN    NaN      NaN
std       NaN      NaN    NaN      NaN
min       NaN      NaN    NaN      NaN
25%       NaN      NaN    NaN      NaN
50%       NaN      NaN    NaN      NaN
75%       NaN      NaN    NaN      NaN
max       NaN      NaN    NaN      NaN
```

**Data Preparation:** .

[6]:
```
#drop the columns that are not needed for the timeseries project and checking␣
 ↪of datatypes
df.
 ↪drop(columns=["sensor_id","sensor_type","location","P1","durP1","ratioP1","durP2","ratioP2"
 ↪inplace=True)
print(df.info())
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58994 entries, 0 to 58993
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   lat        58994 non-null  float64
 1   lon        58994 non-null  float64
 2   timestamp  58994 non-null  object
 3   P2         58994 non-null  object
dtypes: float64(2), object(2)
memory usage: 1.8+ MB
None
```

[6]:
```
      lat     lon                         timestamp   P2
0 -6.818  39.285  2017-12-01T00:00:44.094494+00:00    4
1 -6.818  39.285  2017-12-01T00:03:13.870148+00:00  3.9
2 -6.818  39.285  2017-12-01T00:05:43.505681+00:00  3.9
```

```
 3 -6.818   39.285   2017-12-01T00:08:13.116753+00:00   4.3
 4 -6.818   39.285   2017-12-01T00:10:42.711977+00:00   4.6
```

[7]:
```
#Adjust the datatype of P2 to float and correct an error within by replacing
df['P2'] = df['P2'].str.replace('>', '').astype(float)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58994 entries, 0 to 58993
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   lat        58994 non-null  float64
 1   lon        58994 non-null  float64
 2   timestamp  58994 non-null  object
 3   P2         58994 non-null  float64
dtypes: float64(3), object(1)
memory usage: 1.8+ MB
```

[8]:
```
fig = px.scatter_mapbox(
    df,
    lat="lat",
    lon="lon",
    center={"lat": -6.7924, "lon": 39.2083},  # Centered on Dar es Salaam
    width=600,
    height=600,
    hover_data=["P2"],  # Display price when hovering mouse over house
)

fig.update_layout(mapbox_style="open-street-map")

#fig.show()
#had to comment the command and bring in the png format so as to make it␣
 ↪visible for pdf view
scatter_mapbox_image = Image.open(r"C:\Users\23162342\Desktop\POS␣
 ↪Reports\python timeseries project\newplot.png")
scatter_mapbox_image
```

[8]:




The result shows that the PM2.5 readings were recorded at only two different location. let's check below for the unique locations to confirm our data

[9]:
```
print("unique latitude for reading:",df["lat"].unique())
print("unique longitude for reading:",df["lon"].unique())
```

```
unique latitude for reading: [-6.818 -6.902]
unique longitude for reading: [39.285 39.267]
```

**How bad can the Air quality be?:** .

[10]:
```
Explianer_image = Image.open(r"C:\Users\23162342\Desktop\POS Reports\python␣
↪timeseries project\Air_quality_explainer.jpg")
Explianer_image
```

[10]:

| AQI | Air Pollution Level | Health Implications | Cautionary Statement (for PM2.5) |
|---|---|---|---|
| 0 - 50 | Good | Air quality is considered satisfactory, and air pollution poses little or no risk | None |
| 51 -100 | Moderate | Air quality is acceptable; however, for some pollutants there may be a moderate health concern for a very small number of people who are unusually sensitive to air pollution. | Active children and adults, and people with respiratory disease, such as asthma, should limit prolonged outdoor exertion. |
| 101-150 | Unhealthy for Sensitive Groups | Members of sensitive groups may experience health effects. The general public is not likely to be affected. | Active children and adults, and people with respiratory disease, such as asthma, should limit prolonged outdoor exertion. |
| 151-200 | Unhealthy | Everyone may begin to experience health effects; members of sensitive groups may experience more serious health effects | Active children and adults, and people with respiratory disease, such as asthma, should avoid prolonged outdoor exertion; everyone else, especially children, should limit prolonged outdoor exertion |
| 201-300 | Very Unhealthy | Health warnings of emergency conditions. The entire population is more likely to be affected. | Active children and adults, and people with respiratory disease, such as asthma, should avoid all outdoor exertion; everyone else, especially children, should limit outdoor exertion. |
| 300+ | Hazardous | Health alert: everyone may experience more serious health effects | Everyone should avoid all outdoor exertion |

[11]:
```
#drop lat and lon columns to focus on the time series of pm2.5 readings
df.drop(columns=["lat","lon"],inplace=True)
```

[12]:
```
#convert the string timestamp into datetime type and the p2 to float
df["timestamp"]= df["timestamp"].str.split(".", expand=True)[0]
df['timestamp'] = pd.to_datetime(df['timestamp'])
df.head()
```

[12]:
```
             timestamp   P2
0 2017-12-01 00:00:44  4.0
1 2017-12-01 00:03:13  3.9
2 2017-12-01 00:05:43  3.9
3 2017-12-01 00:08:13  4.3
4 2017-12-01 00:10:42  4.6
```

[13]:
```
#set the timestamp as an index
df.set_index("timestamp",inplace=True)
```

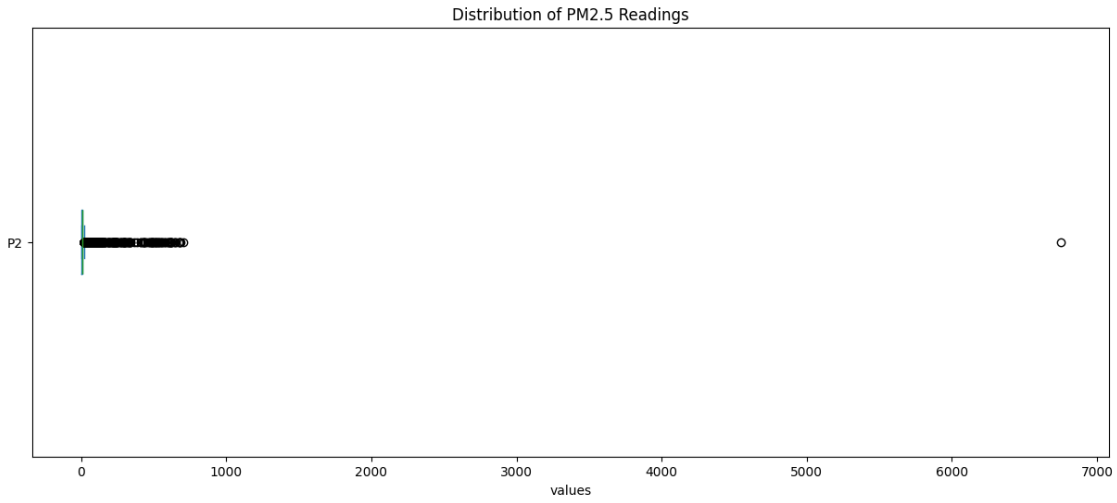
[14]:
```
#make the timestamp the index and convert it to a time in UTC and convert to␣
 ↪locatime in tanzania
df.index=df.index.tz_localize("UTC").tz_convert("Africa/Dar_es_Salaam")
df.head()
```

[14]:
```
                           P2
timestamp
2017-12-01 03:00:44+03:00  4.0
```

```
2017-12-01 03:03:13+03:00  3.9
2017-12-01 03:05:43+03:00  3.9
2017-12-01 03:08:13+03:00  4.3
2017-12-01 03:10:42+03:00  4.6
```

[15]:
```
fig, ax = plt.subplots(figsize=(15, 6))
df.plot(kind="box", vert=False, xlabel="values", title="Distribution of PM2.5 Readings", ax= ax)
```

[15]:
```
<Axes: title={'center': 'Distribution of PM2.5 Readings'}, xlabel='values'>
```

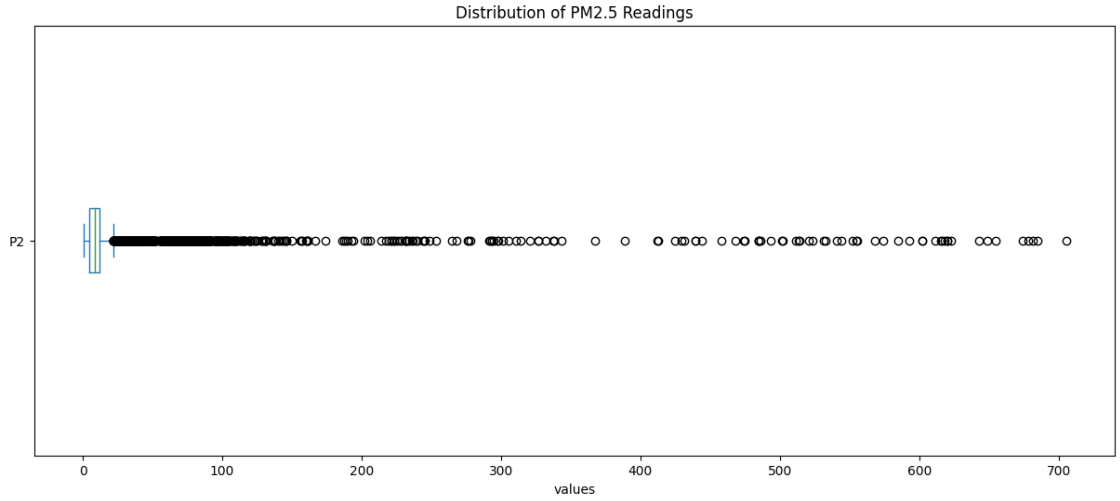



[16]:
```
#remove outliers
df=df[df["P2"]<6747]
df.head()
```

[16]:
```
                           P2
timestamp
2017-12-01 03:00:44+03:00  4.0
2017-12-01 03:03:13+03:00  3.9
2017-12-01 03:05:43+03:00  3.9
2017-12-01 03:08:13+03:00  4.3
2017-12-01 03:10:42+03:00  4.6
```

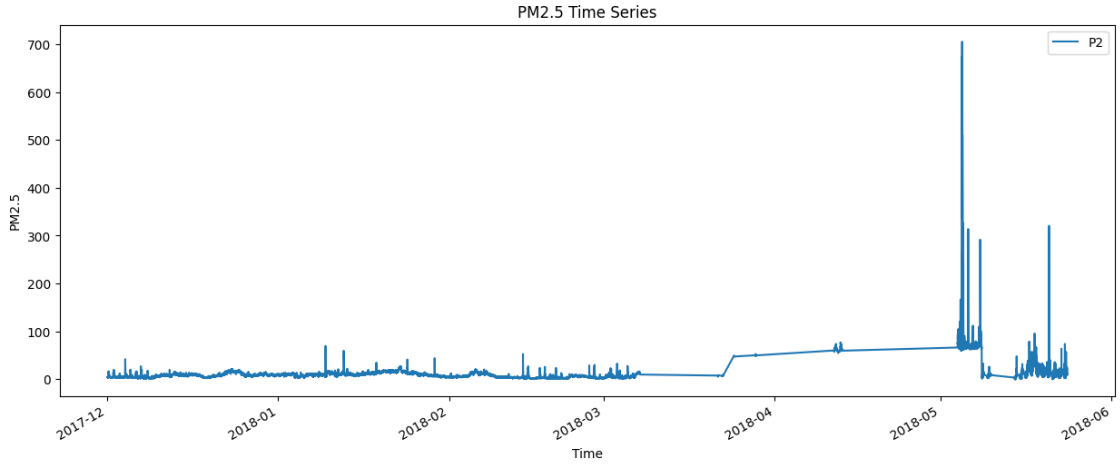
[17]:
```
fig, ax = plt.subplots(figsize=(15, 6))
df.plot(kind="box", vert=False, xlabel="values",title="Distribution of PM2.5 Readings", ax= ax)
```

[17]:
```
<Axes: title={'center': 'Distribution of PM2.5 Readings'}, xlabel='values'>
```




[18]:
```
fig, ax = plt.subplots(figsize=(15, 6))
df.plot(xlabel="Time", ylabel= "PM2.5", title="PM2.5 Time Series",ax=ax)
```

[18]: <Axes: title={'center': 'PM2.5 Time Series'}, xlabel='Time', ylabel='PM2.5'>




[19]:
```
#resample the time index to 1h and fill up missing values with mean.
df= df["P2"].resample("1h").mean().fillna(method="ffill").to_frame()
```

[20]:
```
df.info()
```

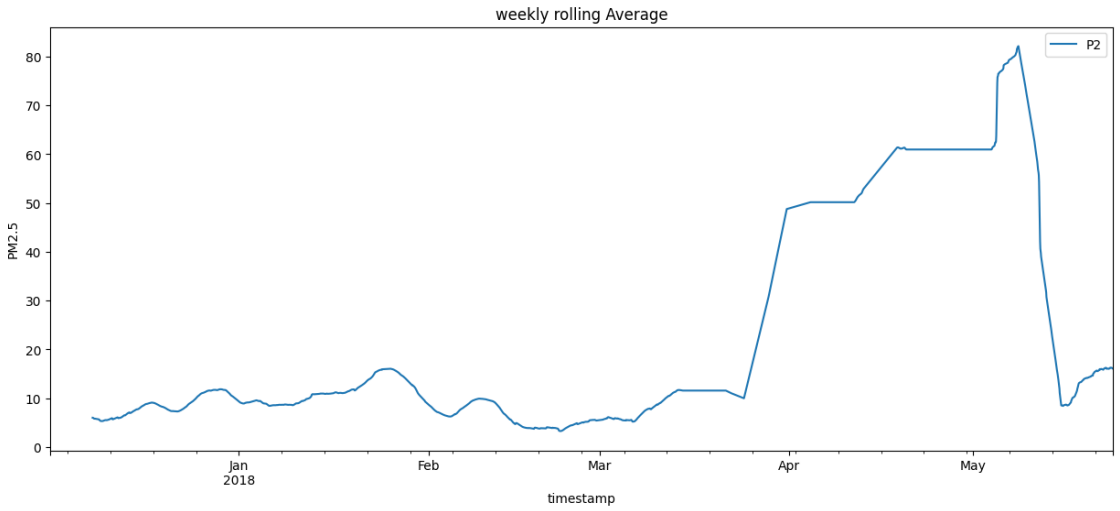
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 4173 entries, 2017-12-01 03:00:00+03:00 to 2018-05-23
23:00:00+03:00
Freq: H
```

```
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   P2      4173 non-null   float64
dtypes: float64(1)
memory usage: 65.2 KB
```

[21]:
```
df["P2"].rolling(168).mean().isnull().sum()
```

[21]:
```
167
```

[22]:
```
#Plot the rolling average of the "P2" readings in df. Use a window size of 168
 ↪(the number of hours in a week).
fig, ax = plt.subplots(figsize=(15, 6))
df.rolling(168).mean().plot(ax=ax, title="weekly rolling Average", ylabel="PM2.
 ↪5");
```




**Data Wrangling:** .
The exploratioin has successfully been made on the dataset and series of adjustment has been made, so to make the process of the adjustment compact and easy to directly understand, the wrangling function is here to help us with that, by compiling the changes and making it effective with just one line of code.

[23]:
```
def wrangle( filepath, RSV="1H"):

    #read the file in a dataframe
    df = pd.read_csv(filepath)

    #drop the columns that are not needed for the timeseries project and
 ↪checking of datatypes
```

```
    df.
↪drop(columns=["sensor_id","sensor_type","location","lat","lon","P1","durP1","ratioP1","durP
↪inplace=True)

    #convert the string timestamp into datetime type and the p2 to float
    df["timestamp"]= df["timestamp"].str.split(".", expand=True)[0]
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['P2'] = df['P2'].str.replace('>', '').astype(float)

    #set the timestamp as an index
    df.set_index("timestamp",inplace=True)

    #make the timestamp the index and convert it to a time in UTC and convert␣
↪to locatime in tanzania
    df.index=df.index.tz_localize("UTC").tz_convert("Africa/Dar_es_Salaam")

    #remove outliers
    df=df[df["P2"]<6747]

    #resample the time index to 1h and fill up missing values with mean.
    df= df["P2"].resample(RSV).mean().fillna(method="ffill").round(2).to_frame()



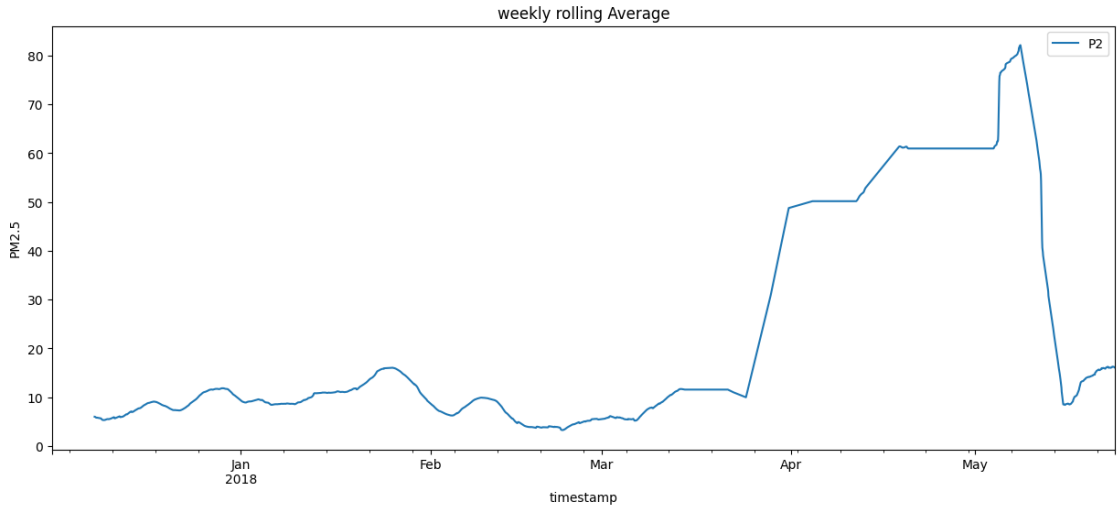
    return df
```

[24]:
```
df = wrangle(r"C:\Users\23162342\Desktop\POS Reports\python timeseries␣
↪project\Temekeairquality.csv")
df.head()
```

[24]:

| timestamp | P2 |
|---|---|
| 2017-12-01 03:00:00+03:00 | 4.20 |
| 2017-12-01 04:00:00+03:00 | 4.34 |
| 2017-12-01 05:00:00+03:00 | 9.38 |
| 2017-12-01 06:00:00+03:00 | 14.06 |
| 2017-12-01 07:00:00+03:00 | 14.54 |

[25]:
```
#Plot the rolling average of the "P2" readings in df. Use a window size of 168␣
↪(the number of hours in a week).
fig, ax = plt.subplots(figsize=(15, 6))
df.rolling(168).mean().plot(ax=ax, title="weekly rolling Average", ylabel="PM2.
↪5");
```

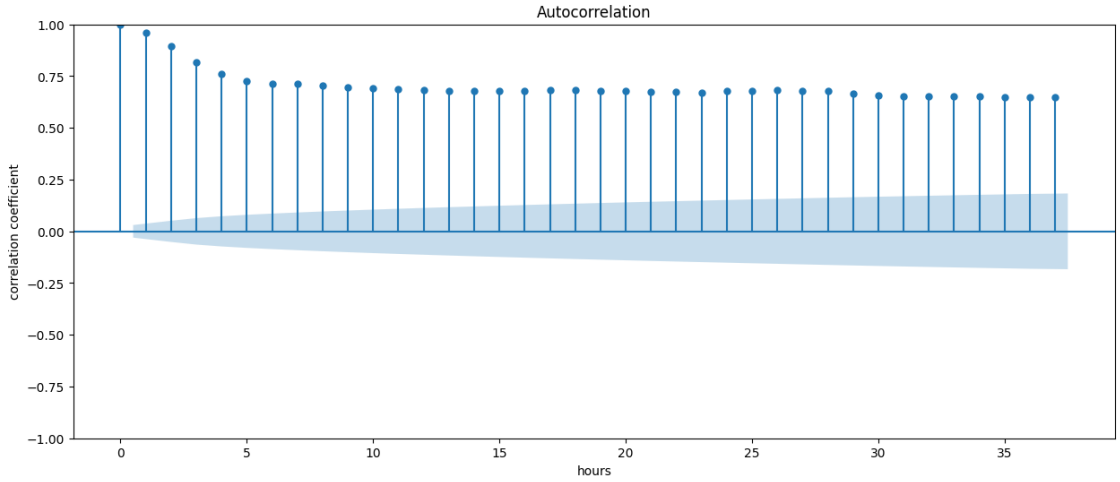



[26]:
```
#change the dataframe into series since its just a column and an index
y=df.squeeze()
y.head()
```

[26]:
```
timestamp
2017-12-01 03:00:00+03:00     4.20
2017-12-01 04:00:00+03:00     4.34
2017-12-01 05:00:00+03:00     9.38
2017-12-01 06:00:00+03:00    14.06
2017-12-01 07:00:00+03:00    14.54
Freq: H, Name: P2, dtype: float64
```

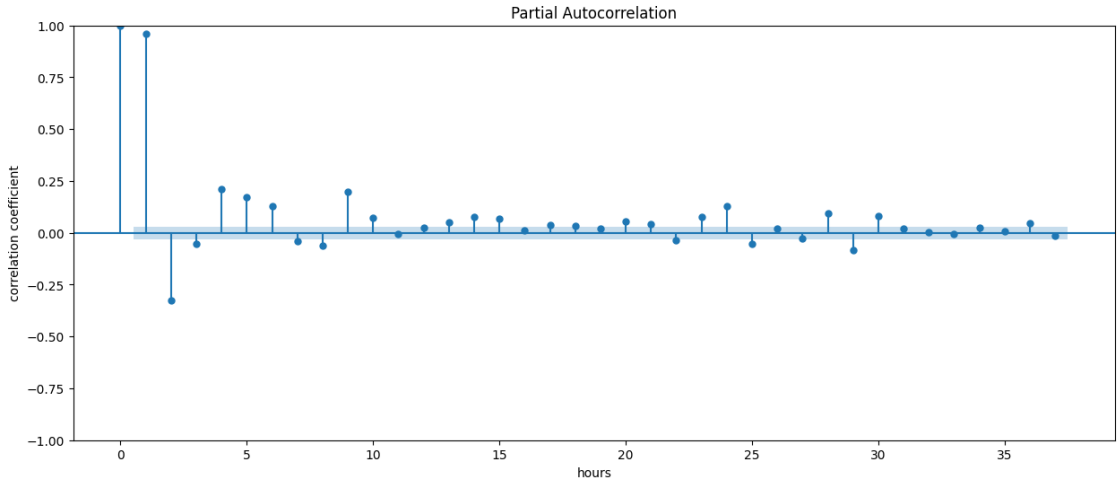
[27]:
```
#PLoting the acf plot
fig, ax = plt.subplots(figsize=(15,6))
plot_acf(y,ax=ax)
plt.xlabel("hours")
plt.ylabel("correlation coefficient")
```

[27]:
```
Text(0, 0.5, 'correlation coefficient')
```




[28]:
```
#PLoting the pacf plot
fig, ax = plt.subplots(figsize=(15,6))
plot_pacf(y,ax=ax)
plt.xlabel("hours")
plt.ylabel("correlation coefficient")
```

[28]:
```
Text(0, 0.5, 'correlation coefficient')
```




[29]:
```
#split y using 99.95% lof it to train and the rest to test
print("total lenght of y:", len(y))
cutoff= int(len(y)*0.994)
print("cutoff point is:", cutoff)

y_train= y.iloc[:cutoff]
```

```
y_test=y.iloc[cutoff:]
print("lenght of training set:", len(y_train))
print("lenght of test set:", len(y_test))
```

```
total lenght of y: 4173
cutoff point is: 4147
lenght of training set: 4147
lenght of test set: 26
```

[30]:
```
y_train_mean=y_train.mean()
y_pred_baseline= [y_train_mean] * len(y_train)
mae_baseline= mean_absolute_error(y_train,y_pred_baseline)



print("Mean P2 Reading:", round(y_train_mean, 2))
print("Baseline MAE:", round(mae_baseline, 2))
```

```
Mean P2 Reading: 22.28
Baseline MAE: 19.66
```

[31]:
```
#Create ranges for possible    and   , and  p_params should range between 0 and␣
 ↪18, by steps of 4. q_params should range between 0 and 3 by steps of 1.
p_params = range(0,26,5)
q_params = range(0,3,1)
```

[32]:
```
# Create dictionary to store MAEs
mae_grid = dict()
# Outer loop: Iterate through possible values for `p`
for p in p_params:
    # Create key-value pair in dict. Key is `p`, value is empty list.
    mae_grid[p] = list()
    # Inner loop: Iterate through possible values for `q`
    for q in q_params:
        # Combination of hyperparameters for model
        order = (p, 0, q)
        # Note start time
        start_time = time.time()
        # Train model
        model = ARIMA(y_train, order=order).fit()
        # Calculate model training time
        elapsed_time = round(time.time() - start_time, 2)
        print(f"Trained ARIMA {order} in {elapsed_time} seconds.")
        # Generate in-sample (training) predictions
        y_pred = model.predict()
        # Calculate training MAE
        mae = mean_absolute_error(y_train, y_pred)
```

```
        # Append MAE to list in dictionary
        mae_grid[p].append(mae)

print()
print(mae_grid)
```

```
Trained ARIMA (0, 0, 0) in 0.38 seconds.
Trained ARIMA (0, 0, 1) in 0.28 seconds.
Trained ARIMA (0, 0, 2) in 0.89 seconds.
Trained ARIMA (5, 0, 0) in 0.67 seconds.
Trained ARIMA (5, 0, 1) in 2.63 seconds.
Trained ARIMA (5, 0, 2) in 3.28 seconds.
Trained ARIMA (10, 0, 0) in 1.96 seconds.
Trained ARIMA (10, 0, 1) in 5.85 seconds.
Trained ARIMA (10, 0, 2) in 7.21 seconds.
Trained ARIMA (15, 0, 0) in 2.65 seconds.
Trained ARIMA (15, 0, 1) in 12.56 seconds.
Trained ARIMA (15, 0, 2) in 12.1 seconds.
Trained ARIMA (20, 0, 0) in 5.93 seconds.
Trained ARIMA (20, 0, 1) in 24.62 seconds.
Trained ARIMA (20, 0, 2) in 24.5 seconds.
Trained ARIMA (25, 0, 0) in 15.81 seconds.
Trained ARIMA (25, 0, 1) in 39.69 seconds.
Trained ARIMA (25, 0, 2) in 40.28 seconds.

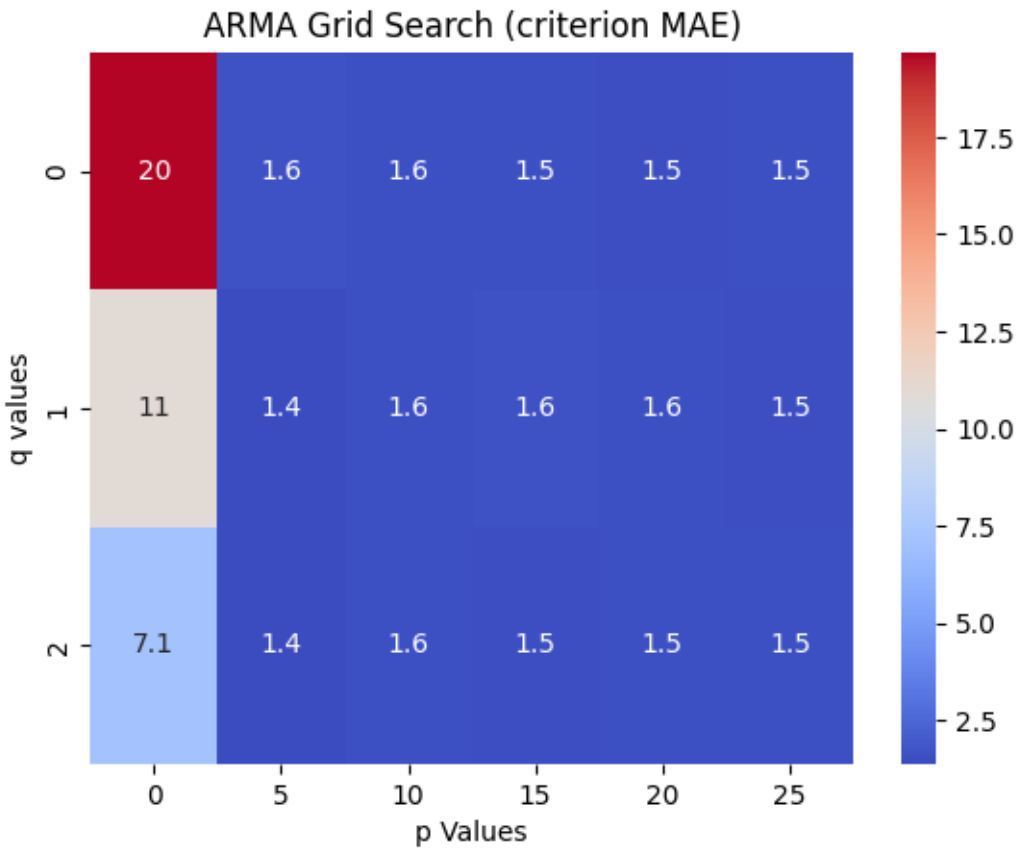
{0: [19.65915358902091, 10.922920525810984, 7.1362297958619365], 5:
[1.6350834503626797, 1.3907952701383928, 1.397285081889126], 10:
[1.5790809930191398, 1.586649748033784, 1.5768617695323124], 15:
[1.548157777574667, 1.6298101729999577, 1.5199378838366266], 20:
[1.518609103387365, 1.5556196985093587, 1.5413283750870175], 25:
[1.537503556504592, 1.4767310074280653, 1.5362370804175032]}
```

[33]:
```
mae_grid=pd.DataFrame(mae_grid)
round(mae_grid,5)
```

[33]:

| | 0 | 5 | 10 | 15 | 20 | 25 |
|---|---|---|---|---|---|---|
| 0 | 19.65915 | 1.63508 | 1.57908 | 1.54816 | 1.51861 | 1.53750 |
| 1 | 10.92292 | 1.39080 | 1.58665 | 1.62981 | 1.55562 | 1.47673 |
| 2 | 7.13623 | 1.39729 | 1.57686 | 1.51994 | 1.54133 | 1.53624 |

[34]:
```
sns.heatmap(mae_grid, annot=True, cmap="coolwarm")
plt.xlabel("p Values")
plt.ylabel("q values")
plt.title("ARMA Grid Search (criterion MAE)");
```

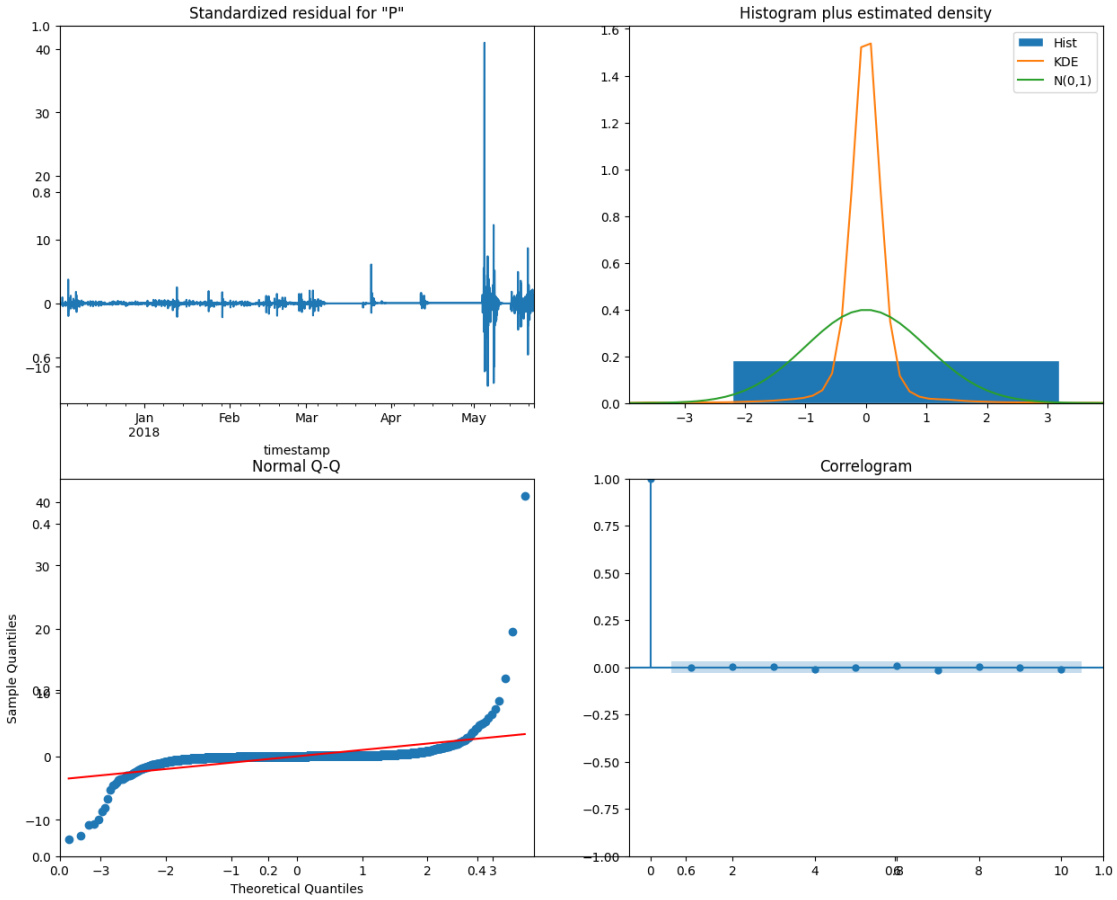


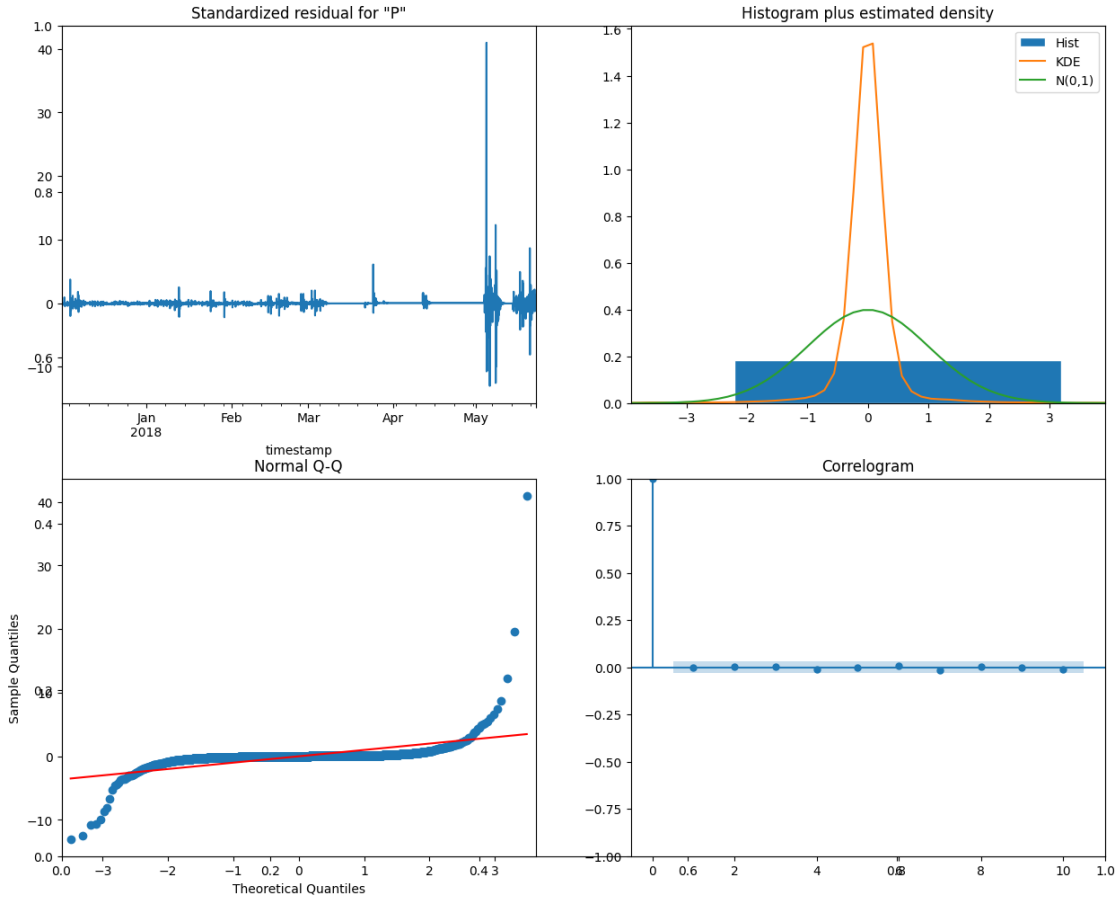

**Data Residual:**

Examining residuals is a crucial step in validating a model. Residuals are the differences between the observed values and the value predicted by the model. 1. Well behaved residual plot will be randomly scattered around the horizontal axis of the residual scattered plot, indicating that the model is well fitted to the data 2. The diagonistic's standardized residual for "p" must range around the 0 axis horizontally 3. The "histogram plus estimated density" must be nicely fitted within the lines and not irregularly. 4. The line on the "Normal Q-Q" must nicely fit the majority od the plot 5. The correlogram's blue shaded range should overshadow the observations on the plot.|

[35]:
```
#Use the plot_diagnostics method to check the residuals for your model.
fig, ax =plt.subplots(figsize=(15,12))
model.plot_diagnostics(fig=fig)
```

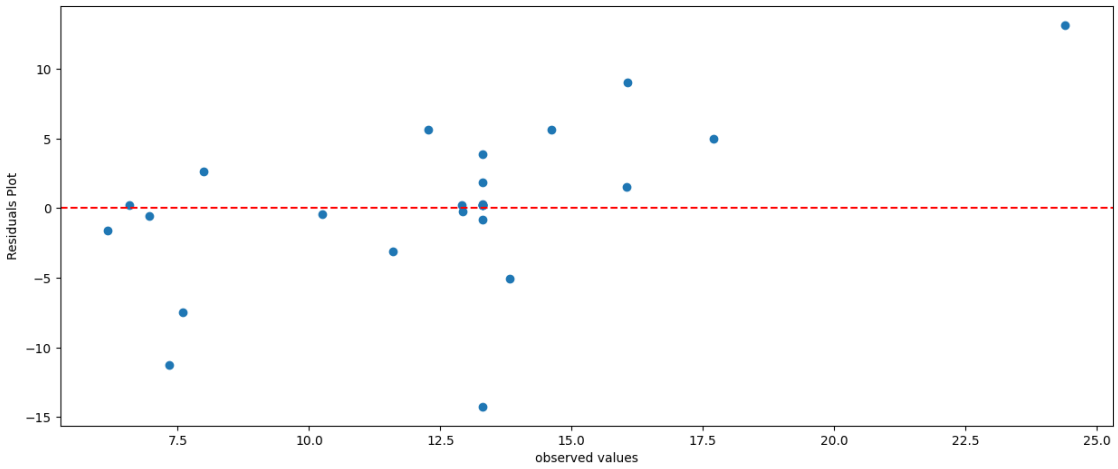
[35]:


Standardized residual for "P"
Jan
2018
Feb
Mar
Apr
May
timestamp
Histogram plus estimated density
Hist
KDE
N(0,1)
Normal Q-Q
Sample Quantiles
Theoretical Quantiles
Correlogram




[37]:
```
y_pred_wfv = pd.Series()
history = y_train.copy()
for i in range(len(y_test)):
    model = ARIMA(history, order=(5,0,1)).fit()
    next_pred = model.forecast()
    y_pred_wfv = y_pred_wfv._append(next_pred)
    history = history._append(y_test[next_pred.index])
```

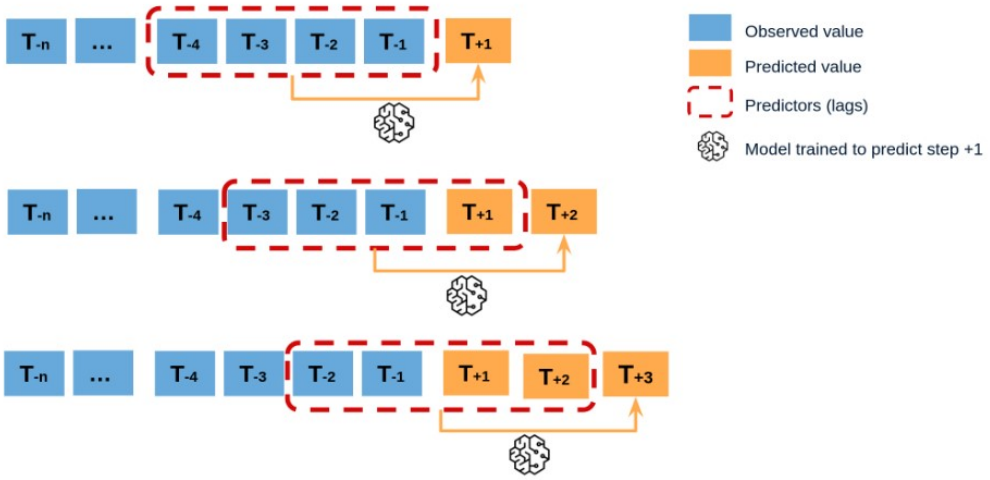
[38]:
```
residual = y_test - y_pred_wfv
fig, ax =plt.subplots(figsize=(15,6))
plt.scatter(y_test, residual)
plt.axhline(y=0, color="r", linestyle="--")
plt.xlabel("observed values")
plt.ylabel("Residuals Plot")
plt.show()
```




[39]:
```
forecast_image = Image.open(r"C:\Users\23162342\Desktop\POS Reports\python timeseries project\Recursive.jpg")
forecast_image
```

[39]:

- Recursive multi-step forecasting




[40]:
```
test_mae = mean_absolute_error(y_test, y_pred_wfv)
print("Test MAE (walk forward validation):", round(test_mae, 2))
```

```
Test MAE (walk forward validation): 3.64
```

**Result Communication:**
Since the Test MAE(mean absolute error) is way very low to the Baseline MAE, it can only means that the model is doing absolutely fantastic and then we can go ahead and use the model for what

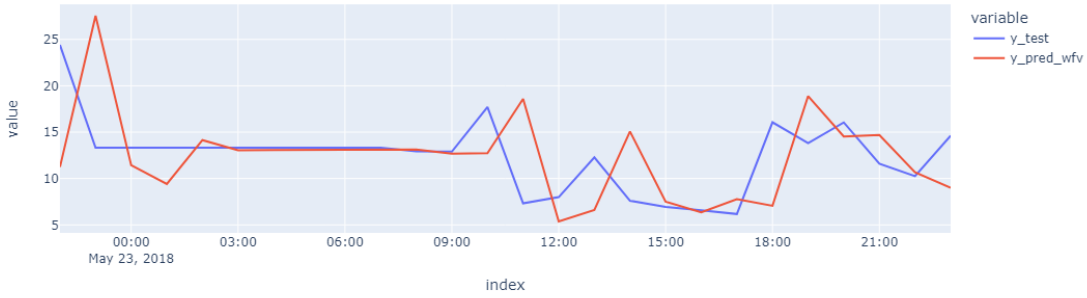
we wish to use it for since the margin in the possible error occurence has been grossly diminished, we can be rest assured of having the best prediction from the model.

[41]:
```
df_predictions = pd.DataFrame({"y_test": y_test, "y_pred_wfv":␣
↪round(y_pred_wfv,2)})
df_predictions.tail()
```

[41]:

| | y_test | y_pred_wfv |
|---|---|---|
| 2018-05-23 19:00:00+03:00 | 13.82 | 18.89 |
| 2018-05-23 20:00:00+03:00 | 16.05 | 14.54 |
| 2018-05-23 21:00:00+03:00 | 11.60 | 14.69 |
| 2018-05-23 22:00:00+03:00 | 10.25 | 10.68 |
| 2018-05-23 23:00:00+03:00 | 14.62 | 9.01 |

[42]:
```
df_predictions = pd.DataFrame({"y_test": y_test, "y_pred_wfv":␣
↪round(y_pred_wfv,2)})
fig = px.line(df_predictions, labels={"values","PM2.5"})
fig.show()
```




**Conclusion:**
Concluding our journey through the Tanzanian air quality time series, our meticulously crafted model showcases a remarkable Mean Absolute Error (MAE) of 3, a significant stride from the initial baseline of 19. This success not only affirms the model's efficacy but also beckons us to explore actionable insights for fostering sustainable air quality practices. As we embrace this triumph, let us propel our findings into real-world impact, forging a path toward a greener and healthier future for Tanzania and beyond.

**About the Author:** Mike is passionate about using data to solve complex problems. With years of experience in the industry, I have honed my skills in data visualization, statistical analysis, and data modeling. My expertise lies in translating complex data into meaningful insights that drive business growth. Whether you're a small business owner or a large corporation, I can help you make sense of your data. Let's work together to uncover insights that will drive your success.
linkedin profile:https://www.linkedin.com/in/micheal-olaniyi/